Kubernetes

14 March 2026 19:38

Definition:

- Kubernetes is an Orchestration Tool.
- Coordinates clusters of computers to work as single unit.
- Designed to automate the deployment, scaling and Management of Containerized Application.
- Originally Developed by **Google** and now maintained by **Cloud Native Computing Foundation (CNCF).**

Core Functions:

- Self Healing: If dies -> Restart new pod/node
- Automated Scaling: Can increase/decrease no.of running containers based on CPU usage or custom metrics.
- Service Discovery & Load balancing: Gives containers their own IP address & Single DNS name for set of containers, then balance traffic across them to ensure stability.
- Storage Orchestration: Auto Mount storage system(local, public cloud etc..) to your application.
- Rollout and Rollbacks: Maintains Desired state always, One at a controlled rate (eg: Updating software version without downtime.)

Architecture:

- Client - Server Architecture. (Control Plane : Master, Node : Worker)

Control Plane:

- Kube-apiserver (The front Door): It act as a brain. The only component that communicate with all others. It exposes the Kubernetes API, Allowing users (via **kubectl**) and internal components to interact with the cluster.
- Etcd (The memory): A highly available key-value store that serves as the cluster's Source of Truth. It stores all the cluster data, including configuration, state, secrets etc...
- Kube-scheduler (The Matchmaker): When new pod is created it decides which worker node it should run on. Make this choice based on resource availability, hardware/software constraints, and specific policies like **affinity rules.**
- Kube-controller-manager (The overseer): This components runs **Controller** processes that constantly watch the state of the cluster through the API server. If the current state doesn't match the desired state (e.g., a node goes down), it trigger corrective action.
  - Node Controller: Manges node health and responds if a node becomes unavailable.
  - Job Controller: Watches for one-off tasks and pods to run them to completion.
- Cloud-controller-manager (The Bridge): An Optional component used when running kubernetes in the cloud (Eg: AWS, Azure, GCP). It links your cluster to the cloud providers API to manage resources like load balancers and storage volumes.

Flow:

- Create Deployment command to the **kube-apiserver.**
- API server validation -> save the **desired state** in **etcd.**
- **kube-controller-manager** notice new deployment in etcd and realize no pods exist yet, so it creates pod objects.
- **kube-scheduler** sees the new pods, check available worker nodes and assigns them to the best node.
- Api server notifies the **kubelet** (on the worker node) to start the containers.

Architecture:

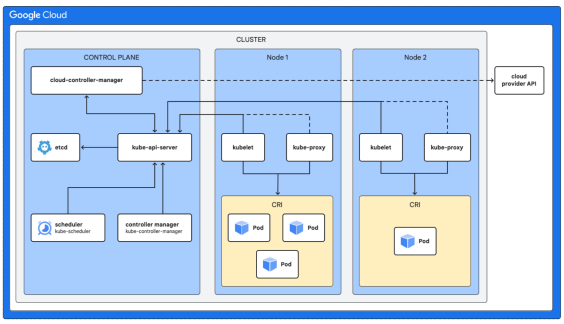
Worker Node:

- Consists of the machines where your applications actually run. While the control plane makes the decisions, the worker plane executes them.
- Kubelet (The Node Agent): Runs on every node in the cluster. Responsible for making sure that containers are running in a Pod as specified in the **PodSpec** (the plan provided by the control plane). If a container crashes, the kubelet is the one that restarts it.
- Kube-proxy (The network Manager): A network proxy that maintains network rules on each node. These rules allow network communication to your Pods from both inside and outside the cluster. It handles simple load balancing by routing traffic to the correct pod within a service.
- Container Runtime Interface (The Engine): The software responsible for actually **running the containers.** Kubernetes supports several runtimes that implement the **CRI**, such as `Containerd, CRI-O, Docker Engine`.

Flow:

- When node starts up, the **kubelet** register it with the API server.
- The kubelet constantly **watches** the API server for any Pods that the **Scheduler** has assigned to that specific node.
- Once task is assigned, the kubelet instructs the **Container Runtime** to pull the necessary images and start the containers.
- The kubelet sends periodic **heart-beats** back to the control plane to confirm the node and its containers are healthy.

Architecture Image:



Basic Yaml Structure:

```
apiVersion: v1 | app/v1 # version of the kubernetes
kind: Pod | ReplicaSet | Deployment ... # object
Metadata: # helps to uniquely identify the object
  name: my-app
  labels:
    env: prod
    any-key: any-value
spec: # source of truth > desired state
  containers:
    - name: container-name
      image: nginx:1.21 # image name
      ports:
        - containerPort: 80
```

Types of kinds:

Category	Kind(s)	Description
Workloads	`Pod`, `Deployment`, `StatefulSet`, `DaemonSet`, `Job`, `CronJob`, `ReplicaSet`	Define how containers are deployed, scaled, and managed.
Networking	`Service`, `Ingress`, `NetworkPolicy`, `IngressClass`, `Gateway`	Manage connectivity, load balancing, and traffic rules.
Config & Secrets	`ConfigMap`, `Secret`	Separate configuration data and sensitive info from container images.
Storage	`PersistentVolume`, `PersistentVolumeClaim`, `StorageClass`	Manage long-term data storage independent of Pod lifecycles.
Security	`Role`, `ClusterRole`, `RoleBinding`,	Control user and workload

Pods:

- Collection of containers, Network, storage etc..
- Every pod has its unique IP address. Communicate via IP address. But **IP address not required to communicate** when containers are in the **same pod**.
- If one container dies in a pod **kubelet** will trigger the self healing (kill the pod -> recreate it), even though we have remaining good containers in the same pod.
- Good Practice is 1 Pod = 1 Container. If there is a requirement to have tightly coupled containers then 1 Pod = Many containers.

pod.yml

- Basic Pod Creation structure.

Secrets		Sensitive info from container images.
Storage	`PersistentVolume`, `PersistentVolumeClaim`, `StorageClass`	Manage long-term data storage independent of Pod lifecycles.
Security (RBAC)	`Role`, `ClusterRole`, `RoleBinding`, `ClusterRoleBinding`, `ServiceAccount`	Control user and workload permissions within the cluster.
Cluster Management	`Namespace`, `Node`, `ResourceQuota`, `LimitRange`, `Event`	Handle the logical and physical boundaries of the cluster.
Autoscaling	`HorizontalPodAutoscaler` (HPA), `VerticalPodAutoscaler` (VPA)	Automatically scale resources based on demand.

pod.yml

- Basic Pod Creation structure.

Command:

- kubectl apply -f pod.yml
- Kubectl get pods
- Kubectl exec -it <pod-name> -- /bin/sh # Opening pod terminal in interactive mode
- Kubectl delete pod <pod-name>
- Kubectl pods -o wide
- Kubectl describe pod <pod-name/id> # step by step exec can be seen under Event portion
- Kubectl get pods --watch # Watch the pods in live

ReplicaSet: (rs)

- It ensure specified no.of of identical pod replica are running at all the times.
- It automatically creates the pod when existing pod crashed/deleted to maintain the desired state.
- 2 ways to increase the pods count.
  - **Imperative**: Increase count using the below command.
    Kubectl scale replicaset <rs-name> --replicas=10
  - **Declarative**: Increase the size in Yml file. (**Best Practice**)

Commands:

- Kubectl get rs
- Kubectl delete rs

```yaml
# ReplicaSet.yml
apiVersion: v1
kind: ReplicaSet
metadata:
  name: my-replicaset
spec:
  replicas: 3
  selector: # Controller -> Manage particular pod
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-container
        image: nginx
        ports:
        - containerPort: 8080

      resources:
        requests:
          memory: "64Mi"
          cpu: "256m"
        limits:
          memory: "128Mi"
          cpu: "512m"
```

Deployment:

- High level resource object that provides declarative updates for Pods and Replicaset.
- Describe "Desired state" for your app in a yaml file, **Deployment Controller** works to change the actual state to that desired state.
- It provides: Self-Healing, Automated Rollouts (Replace old pod with new without downtime), Easy Rollback (Previous version of app), Scaling (inc/dec no.of running instances based on traffic.)
- Deployment Strategies:
  - Rolling Update: Replace old pod with new one by one without downtime.
  - Recreate: Terminate all old pods and recreating new with downtime.
  - Canary: Deploy new version to small subset of users to test stability before full rollout.

Command:

- Kubectl rollout undo deployment <deployment-name> # Rollout to previous version.
- Kubectl rollout history deployment <deployment-name>

```yaml
# deployment.yml
apiVersion: app/v1
kind: Deployment
metadata:
  name: my-deployment
  label:
    app: my-app
spec:
  replica: 3
  strategy:
    type: RollingUpdate
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx
          ports:
            - containerPort: 8080
```

Service: (Stable IP address)

- It helps pod-to-pod and user-to-pod communication.
- Since Pods are Ephemeral (Die and get replaced with new Ip addresses), A **service provide a stable IP address** and **DNS name** so other apps can find them reliably.
- If service created -> **End point** will also created automatically.
- End point maintains the Pod's IP address and provide this IP address to Service.
- **Flow:** Service **->** kube-proxy **->** End points verification (End point Controller always update the Pods IP in End point based on selector) **->** kube-proxy send request to pods. (Eg: {service IP : [Endpoint Ips...]})

- **Types:**
  - **ClusterIP (Default)**: Expose service on an **Internal IP** with in cluster. Use this for communication between Internal services (Eg: frontend to database)
  - **NodePort**: Expose service on specific port (30000 - 32767) on every **Node's IP.** Can access from outside the cluster by hitting <NodeIP>:<NodePort>
  - **LoadBalancer**: Used in cloud env (GCP, Azure, AWS). It automatically create a cloud load balancer that routes external traffic to your NodePort/ClusterIP
  - **ExternalName:** Maps a service to DNS name (like my.database.example.com) instead of using selectors. It's like CNAME record.

- **KubeProxy:**
  - A network proxy that **runs on every worker node** in your cluster. Maintains **IP-Table**
  - Responsible for implementing the **Kubernetes Service** concept by managing the network rules that allow communication to your Pods.
  - This object maintains the Service IP and End Points IP in a table.
  - **Core Responsibility:**
    - Service Discovery: Watches Service and EndpontSlice object changes.
    - Rule Management: Kube-proxy translates service's high-level definition into low-level network

```yaml
# Service.yml

apiVersion: v1
kind: Service
metadata:
  name: my-Service
spec:
  selector:
    app: my-app # This MUST match the Pod label

  ---

  type: ClusterIP # If `None` then it is Headless service
        # All the request go to one DB to avoid mis-communication
  ports:
    - protocol: TCP # Service port
      port: 80
      targetPort: 8080 # Target Pod port
  ---

  type: NodePort
  ports:
    - port: 80 # Service port
      targetPort: 8080 # Target Pod port
      nodePort: 30000 # Opens a port on all nodes in the cluster
        # Node port only for testing purpose (range 30000 - 32767)
  ---

  type: LoadBalancer # Cloud load balancer will created
  ports:
```

allow communication to your Pods.
- This object maintains the Service IP and End Points IP in a table.
- **Core Responsibility:**
  - Service Discovery: Watches Service and EndpontSlice object changes.
  - Rule Management: Kube-proxy translate service's high level definition -> Low level network rules (NAT) on the node.

Components: Service, End Point, Kube-proxy, End point Controller.

Commands:
- Kubectl get services

```yaml
    # Node port only for testing purpose (range 30000 - 32767)
---

type: LoadBalancer # Cloud load balancer will created
ports:
  - port: 80 # Service port
    targetPort: 8080 # Target Pod port

---

type: ExternalName
externalName: api.weather.com
ports:
  - port: 80
```

ConfigMap:

- This object used to store **non-sensitive** configuration data in key-value pairs.
- Keep application code separate from its configuration.
- This helps to use same container in Different phases like Dev, Stg, Prod.
- **Two ways** to do this:
  - As Environment Variable : put Env to -> pod's Env
  - As File (Mounted Volume)
    - Directly Using CM : This Creates n Env = n files (under specified loc)
    - From Yaml Env : n Env in 1 yml File = 1 File (**Best Practice**)
- **Rules**:
  - Not Encrypted
  - Can not exceed 1 MiB
  - When using Mounted Volume -> Eventually update automatically.
  - When using Environment variable -> Pod must be restarted to see the changes.

Command:
- Kubectl apply -f config_map.yaml
- Kubectl create configmap <config_map_name> --from-file="<file.yaml>"

Environment: -> envFrom: [confiMapRef: {name: db-config}]

```yaml
# Config_map.yaml
apiVersion: v1
Kind: ConfigMap
metadata:
  name: db-config
data:
  db-name: my-db
  db-user: my-user
  db-password: [PASSWORD]
  db-host: localhost
```

Deployment Files for Config Map From Env

```yaml
# Config_map.yaml
apiVersion: v1
metadata:
  name: simplybyte-configmap
data:
  db-name: my
```

Deployment File for CM from File:

```yaml
# Config_map.yaml
```

Same as left side : This creates n Env = n Files

Application.yml

Page:
title: Title From Yml File

- Kubectl create configmap <cm-name> --from-file=Application.yml

Deployment File is Same as above just change the ConfigMap name to <cm-name>

Secrets:
- This object designed to store **Sensitive Information** such as password, Token etc..
- By default, Kubernetes stores secrets unencrypted in **etcd.**
  - **Enable Encryption at Rest** in your cluster configuration.
  - **Use an External KMS (Key Management Service**) like AWS KMS, Azure Key Vault or HashiCorp vault
- Characteristics:
  - Data Stored as **Base64 string** (this is obfuscation not encryption)
  - By default, Secret are stored in **tmpfs (RAM)** on the worker nodes, they are not written to physical disk.
  - Can use RBAC to limit which users or Pods can see them.

Using Manifest yml:

```yaml
# Secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-db-secret
type: Opaque
data:
  db-user: bXktdXNlcg==
  db-password: cGFzc3dvcmQK
  db-host: bG9jYWxob3N0
```

Application.yml

Page:
title: aGVsbG8gVGhhbXUK

Deployment File for Secret:

2. The Pod/Deployment with Volume Mount

The `volumes` section defines the source, and `volumeMounts` defines where it appears inside the container.

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
  - name: my-app
    image: nginx
    volumeMounts:
    - name: secret-volume
      mountPath: "/etc/secrets"  # Folder where files will appear
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: my-db-secret   # Must match the name in metadata above
```

yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-config
type: Opaque
stringData: # Use stringData to avoid manual Base64 encoding!
  DB_URL: "postgresql://admin:p@ssword123@db-host:5432/mydb"
```

Use code with caution.

The Pod (Using it):

yaml

```yaml
spec:
  containers:
  - name: backend
    image: my-app-image
```

Application.yml

Page:
title: aGVsbG8gVGhhbXUK

- Kubectl create Secret <cm-name> --from-file=Application.yml
- Kubectl get Secret

```yaml
spec:
  containers:
  - name: backend
    image: my-app-image
    envFrom:                # This pulls ALL keys from the secret as env vars
    - secretRef:
        name: db-config
```

Deamon Set:

- Usually **one Pod per Node**. It is specific type of workload controller that ensure a **copy of particular Pod runs on every (or selected) node** within cluster.
- Primary Use case: Deploy background services or system-level tasks that need to be consistently present across the entire infrastructure.
- Eg: Log Collection (to gather log from all the node), Cluster Monitoring (Prometheus..), etc..

- **kind : DaemonSet.** This is the only change we need to do remain everything is same as Deployment.

Persistent Volume (PV) and PVC & Storage class:

- Work together to manage storage independently of a Pod's lifecycle, Ensure that **data is not lost** when Pod is deleted or rescheduled.
- Pod uses PVC and the cluster find the bound PV and mount it to the Pod's container at a specified path.
- If PVC deleted the bound PV is released.
- **Persistent Volume:**
  - Cluster level storage. Physical storage (like NFS or Cloud Disk).
  - Provisioned by **Admin** or Dynamically via **StorageClass**.
  - Access Modes:
    - ReadWriteOnce - Read-write by Single Node
    - ReadOnlyMany - Read-only by Many Node
    - ReadWriteMany - Read-Write by Many node
    - ReadWriteOncePod - Read-write by single Pod only.
- **Persistent Volume Claim (PVC):**
  - Request for storage by User or Application from PV. Namespace Level.
  - PVC consumes **PV resource based on specific requirement** like size and access mode.
- **Storage Class:**
  - An abstraction layer that allows **Dynamic Provisioning**, where PV is automatically created based on PVC request.
  - **Reclaim Policy**
    - Retain: PV and its data are kept for manual recovery.
    - Delete: PV and The underlying storage asset (AWS EBS) are deleted.
    - Recycle: (Deprecated) Scrubs the data and make it available for new claim.

Command:

- Kubectl apply -f <pv | pvc | sc . yml>
- Kubectl get <pv | pvc | sc >

```yaml
# persistent_volume.yml

apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /data/test
```

```yaml
# StorageClass.yml

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sto-class
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
  # Delays until a Pod is scheduled
allowVolumeExpansion: true
  # Allows resizing the volume later
```

```yaml
# pvc.yml

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: sto-class
    # this only when using SC
```

```
# this PVC will automatically bind
with pv based on specified
requirement.
```

Pod Config:

```yaml
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - mountPath: "/data"
          name: storage
  volumes:
    - name: storage
      persistentVolumeClaim:
        claimName: mypvc
```

StatefulSet:

- Workload object designed to manage stable names for Pods.
- Each Pod in a statefulSet is unique and persistent (unlike Deployment, Interchangeable and anonymous)
- For each **Pod will have their dedicated VolumeClaim** Dynamically.
- Stateful Application: **# StatefulSet helps this.**
  - Save data to Local or Persistent and need that exact data back if they restart. **Ex:** Databases, Message Queue, Distributed System.
  - Self-healing required careful orchestration. Strict order for scaling (0 -> 1 -> 2).
- Stateless Application:
  - Treat Every request as a completely new event. They don't store data locally. **Ex:** Web Frontend, Rest API, Microservices.
  - Self-Healing is simple, Random/parallel scaling,
  - Anonymous Pod Identity.

- Characteristics:
  - **Stable Network ID**: Pod are named like [name]-[index] (Eg: db-0, db-1). This name is same even if the Pod is rescheduled to different node.
  - **Ordered Deployment:** Pods are created, updated and deleted in a specific order (0, 1, 2 ....)
  - **Stable Storage:** Each Pod gets it own dedicated **PV** that follows it if the Pod restrarts.
- **Advantage**:
  - Data Integrity: Ideal for DBs(MySql, ..) because data is tied to a specific Pod Identity.
  - Predictable Networking, Safe Updates.

Manifest Files:

```yaml
# Serive.yml

apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None
    # Required for Headless Service
  selector:
    app: mysql # Pod Name
```

```yaml
# StatefulSet.yml

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql-db
spec:
  serviceName: "mysql-network"
    # Connects to a specific network setup
  replicas: 3
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:
        app: database
    spec:
      containers:
      - name: mysql
        image: mysql:8.0

  # THE MAGIC: Every Pod gets its own 10GB drive automatically!
  volumeClaimTemplates:
  - metadata:
      name: database-storage
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 10Gi
```

- Data will be persistent across all the Pods.

- **Disadvantage**:
  - Complexity, Slower Scaling, Storage Costs

Liveness, readiness & Startup Probes:

- Without probe Kubernetes only knows if the container process is running.
- Container may running but the application may broken. In-order to identify the application status we need Probes.

Startup Probe:
- "Wait for it.." -> Used where application takes a long time to start up.
- It **disable** Liveness and Readiness check **until the app is finished starting**.

Readiness Probe:
- "Are you ready for customers.." -> Determines if the container is ready to handle the traffic.
- If this probe fails, Kubernetes **Remove the Pod from the Service**. The Pod stays alive, but no traffic is sent to it.

Liveness Probe:
- "Are you still alive.." -> Determines if the Container needs a reboot.
- If this probe fails, Kubernetes **kills the container and recreate it**.

Extra Key-pair:

initialDelaySeconds: 10 # Call API after these period
periodSeconds: 5 # how frequently Call the API
failureThreshold: 3 # Recreate after n times failure.

Example:

```yaml
spec:
  containers:
  - name: my-app
    image: my-app:v1
    ports:
      containerPort: 5000

    volumeMounts:
      - name: anything
        mountPath: /usr/share/

    # 1. Startup: Give it up to 30 seconds to start
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30
      periodSeconds: 1

    # 2. Liveness: Check every 10 seconds if it's frozen
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      periodSeconds: 10

    # 3. Readiness: Only send traffic if /ready returns 200 OK
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      periodSeconds: 5

  Volumes:
    - name: anything
      persistentVolumeClaim:
        claimName: mypvc
```

Ingress and Ingress Controller:

- Ingress is an API object that manages external access to service within cluster, Typically via HTTP and HTTPS. (Access application from Outside.)
- Is also act as **Router** that provide routing rules (**URL paths or Hostnames**) to **direct** the incoming traffic to the **correct internal Service**.
- **Flow:** Request Load -> Load Balancer -> Ingress -> Routing (based on conditions) -> service -> Pods/containers.
- Ingress Rules: Best Practice : Use both rules
  - Path based Routing : Path-Type (**Prefix** Eg: %word%, **Exact** Eg: %word)
  - Host Based Routing : Domain/sub-domain based (Eg: /facebook.com/add-user)
- Components:
  - **Ingress Resource:** YAML configuration (Rules).
  - **Ingress Controller:** The Actual Software (Gateway API, Nginx - Deprecated, Traefik, HAProxy) that implements those rules by acting as a **reverse proxy** and Load balancer.
  - Default Backend: A fallback service that handles requests that don't match any defined rules.
- Benefits:
  - Consolidated Routing: Many service = 1 Ingress (instead of many Cloud LoadBalancer)
  - Path based (/example) and Name-based Virtual Hosting (foo.com).
  - SSL/TLS Termination: Centrally manage security certificates and decrypt traffic at Ingress level before it hits Pod.

Command:
- Kubectl apply -f <ingress.yaml>
- Kubectl get svc -A
- Kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx /controller-v1.9.6/deploy/static/provider/cloud/deploy.yaml # Run Nginx controller

Manifest files:

ingress.yml

Users > apple > Desktop > Kubernetes > simplybyte-app > Ingress > ingress.yml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - simplybyte.com
      secretName: app-tls

  defaultBackend:
    service:
      name: simplybyte-frontend
      port:
        number: 80

  rules:
    - host: simplybyte.com
      http:
        paths:
          - path: /backend
            pathType: Prefix
            backend:
              service:
                name: simplybyte-backend
                port:
                  number: 8090

          - path: /frontend
            pathType: Prefix
            backend:
              service:
                name: simplybyte-frontend
                port:
                  number: 80
```

RBAC: Role Based Access Control:

- Primary mechanism for regulating **access** to **cluster resources** based on **roles** of individual **users** or **service accounts**.
- RBAC built on four primary API objects :- Role, ClusterRole, RoleBinding, ClusterRoleBinding
- Pod **Cannot** access other Pod directly, So we need **serviceAccount**
- Components:
  - Role:
    - Scope: Namespace
    - Set of Permission (verbs + resources) with in a single namespace.
  - RoleBinding:
    - Scope: Namespace
    - Grant the permissions defined in a Role (or ClusterRole) to specific Subject within a namespace.

Role and RoleBinding:

Roles

In Kubernetes, a Role is a declaration of the operations that can be performed on a type of Kubernetes resource within a particular namespace. A simple example of a Role could be: You might need to do some pre-checks as below:

Roles. yaml: This Role, named 'pod-reader', provides read-only access to Pods in the default namespace.

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

RoleBindings

It actually grants the permissions defined in a Role to a user or set of users. It holds a list of subjects and a reference to the Role being granted. Here's an example of a RoleBinding:

RoleBindings.yaml: In this RoleBinding, the 'pod-reader' Role is assigned to the user 'john'. This means that 'john' can now read Pods in the default namespace.

```yaml
kind: RoleBinding
```

Cluster and ClusterRoleBinding:

ClusterRoles

ClusterRole is similar to Role. However, while a Role is namespace-specific, a ClusterRole is not. This means that you can define permissions that apply across all namespaces in your cluster. Here's an example of a ClusterRole.

ClusterRoles.yaml: In this ClusterRole, reading Nodes across the cluster is allowed.

```yaml
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "watch", "list"]
```

ClusterRoleBindings

Similar to a RoleBinding, a ClusterRoleBinding grants the permissions defined in a ClusterRole to a set of users, but it applies cluster-wide.

ClusterRoleBindings.yaml: In this ClusterRoleBinding, the 'node-reader' ClusterRole is assigned to the user 'john', allowing 'john' to read Node information across the entire cluster.

```yaml
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
```

namespace.
    - RoleBinding:
        - Scope: Namespace
        - Grant the permissions defined in a Role (or ClusterRole) to specific Subject within a namespace.
    - Cluster Role:
        - Scope: Cluster
        - Define permission for cluster-scoped resources (Eg. Nodes, PV ..) or Namespaced resources across all namespace
    - ClusterRoleBinding:
        - Scope: Cluster:
        - Grant Permissions defined in a ClusterRole to specific subjects across the entire cluster.
- Permission Rules:
    - Verbs: get, list, create, update, patch, watch, delete
    - Resources: pods, services, secrets, nodes
    - API Groups: -

Command:
- Kubectl auth can-i
- Kubectl exec -it <ser-acc-pod> -- /bin/sh # can access other pods

RoleBindings

It actually grants the permissions defined in a Role to a user or set of users. It holds a list of subjects and a reference to the Role being granted. Here's an example of a RoleBinding:

RoleBindings.yaml: In this RoleBinding, the 'pod-reader' Role is assigned to the user 'john'. This means that 'john' can now read Pods in the default namespace.

```yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ClusterRoleBindings

Similar to a RoleBinding, a ClusterRoleBinding grants the permissions defined in a ClusterRole to a set of users, but it applies cluster-wide.

ClusterRoleBindings.yaml: In this ClusterRoleBinding, the 'node-reader' ClusterRole is assigned to the user 'john', allowing 'john' to read Node information across the entire cluster.

```yaml
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-nodes
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

Role & Service Account:
(Pod-to-Pod communication)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-kavalan-role
  namespace: default
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pod-kavalan-sa
  namespace: default
```

Role Binding For Service Account:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-kavalan-role-binding
  namespace: default
subjects:
  - kind: ServiceAccount
    name: pod-kavalan-sa
    namespace: default
roleRef:
  kind: Role
  name: pod-kavalan-role
  apiGroup: rbac.authorization.k8s.io
```

Pod Deployment File:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pod-kavalan-deployment
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pod-kavalan
  template:
    metadata:
      labels:
        app: pod-kavalan
    spec:
      serviceAccountName: pod-kavalan-sa
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
        command: ["sleep", "3600"]
```

Taints and Tolerations & Node selector and Node Affinity:

Taints & Tolerations:
- Its a mechanism used to control which pods can be scheduled onto which nodes.
- Node Affinity "Attract" pods to specific nodes.
- Taints "Repel" a set of pods.
- Taints : Rule (Lock) No one can be work here without key.
- Tolerations : Can work because it has Key or Special Passes.

Taints Effects:
- NoSchedule: A strict requirement. Won't disturb the already running pods.
- PreferNoSchedule: If requirement not matched, Try to avoid but allows when no other option.
- NoExecute: Most aggressive. It kills the existing running pods if requirement not matches.

Node Selector:
- Tell Pods to schedule onto only defined Node.
- Step 1: Need to add some label to the Node. {cpu: dummy}
- Step 2: Pass this label in Pod's Spec.
- It is an old Legacy scheduling Mechanism. **Can only tell -> matching labels**. Can not define Schedule in not matching label and cannot choose multiple labels etc..

Node Affinity:
- Born to solve the Node selector issues. We can choose multiple ways of scheduling the pods in Node.
    - requiredDuringSchedulingIgnoreDuringExecution: - Pods scheduled in the Node when Conditions met otherwise won't schedule it.
    - preferredDuringSchedulingIgnoreDuringExecution: - Pods scheduled in the Node when Conditions met otherwise schedule in other pods.

Commands:
- Kubectl taint nodes <node-name> **power=Dummy**(Any key : Value): NoSchedule

Example:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-app-deployment
  namespace: default
spec:
  replicas: 4
  selector:
    matchLabels:
      app: python-app
  template:
    metadata:
      labels:
        app: python-app
    spec:
      tolerations:
      - key: "power"
        operator: "Equal"
        value: "Dummy"
        effect: "NoSchedule"
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
        command: ["sleep", "3600"]
```

Node Affinity and Node Selectors:

```yaml
spec:
  tolerations:
  - key: "power"
    operator: "Equal"
    value: "Dummy"
    effect: "NoSchedule"
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: cpu
            operator: In
            values:
            - Dummy
  containers:
  - name: kubectl
    image: bitnami/kubectl:latest
    command: ["sleep", "3600"]
```

```yaml
      app: python-app
    spec:
      nodeSelector:
        cpu: Dummy
      tolerations:
      - key: "power"
        operator: "Equal"
        value: "Dummy"
        effect: "NoSchedule"
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
```

Pod Affinity and Pod Anti-Affinity:
- While Taints and Toleration repel pods from nodes, Pod affinity and Anti-Affinity allow you to constrain **which nodes your pod is eligible** to be scheduled on based on the labels of other pods already running on those nodes.
- Pod Affinity (Attraction):
    - Tells the scheduler to locate a new pod on the same node (or within the same topology) as other pods that match a specific label.
    - **Example:** Communication happen faster when pods are in the same node. So, placing web server and Redis cache in same node. "Pod affinity ensure web server and Redis Cache scheduled on same node".
- Pod Anti-Affinity (Repulsion):
    - Avoid scheduling the new pods on the same node (or topology) as pods with specific labels.
    - **Example:** 3 replica DB, Using Pod-Anti affinity we can ensure no 2 replica run on same Node.
- Types of Implementation:
    - requiredDuringSchedulingIgnoreDuringExecution: - Pod will not be scheduled if the rule isn't met.
    - preferredDuringSchedulingIgnoreDuringExecution: - The scheduler will try to

Pod Affinity Example:

```yaml
spec:
  replicas: 4
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      affinity:
        podAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - redis
            topologyKey: kubernetes.io/hostname
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
        command: ["sleep", "3600"]
```

same Node.

- Types of Implementation:
  - requiredDuringSchedulingIgnoreDuringExecution: - Pod will not be scheduled if the rule isn't met.
  - preferredDuringSchedulingIgnoreDuringExecution: - The scheduler will try to follow it but will still schedule the pod elsewhere if necessary.
  - topologyKey: Common values include
    - Kubernetes.io/hostname - to match specific nodes.
    - Topology.kubernetes.io/zone - to match entire availablity zones

```
            - redis
          topologyKey: kubernetes.io/hostname
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
        command: ["sleep", "3600"]
```

PodAntiAffinity

```
metadata:
  labels:
    app: java-app
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: app
            operator: In
            values:
            - redis
        topologyKey: kubernetes.io/hostname
  containers:
```

Resource Limit & Resource Request:

Resource Limit:

- Kubernetes resource limit is the maximum amount of CPU and memory a container is allowed to use.
- Container will be restarted when reach the max resource limit.

Resource Request:

- It is the minimum amount of CPU and memory a container is guaranteed to get.
- Scheduled the pod when Node has enough resource.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app-deployment
  namespace: default
spec:
  replicas: 4
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
      - name: kubectl
        image: bitnami/kubectl:latest
        command: ["sleep", "3600"]
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```